

Remove tuple-like protocol support from fixed-extent span

Document #: P2116R0
Date: 2020-02-14
Project: Programming Language C++
LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

§ 1 Introduction

In response to [\[LWG3212\]](#), this paper provides wording to remove the support for tuple-like protocol from fixed-extent spans, as directed by LEWG in Prague. This [resolves](#) [\[LWG3212\]](#).

§ 2 Wording

This wording is relative to [\[N4849\]](#).

Edit 20.5.6 [\[tuple.helper\]](#) p6 and p8 as indicated:

```
template<class T> struct tuple_size<const T>;  
template<class T> struct tuple_size<volatile T>;  
template<class T> struct tuple_size<const volatile T>;
```

[...]

- 6 In addition to being available via inclusion of the `<tuple>` header, the three templates are available when any of the headers `<array>` (22.3.2 [\[array.syn\]](#)), `<ranges>` (24.2 [\[ranges.syn\]](#)), `<span>` (22.7.2 [\[span.syn\]](#)), or `<utility>` (20.2.1 [\[utility.syn\]](#)) are included.

```
template<size_t I, class T> struct tuple_element<I, const T>;  
template<size_t I, class T> struct tuple_element<I, volatile T>;  
template<size_t I, class T> struct tuple_element<I, const volatile T>;
```

[...]

- 8 In addition to being available via inclusion of the `<tuple>` header, the three templates are available when any of the headers `<array>` (22.3.2 [\[array.syn\]](#)), `<ranges>` (24.2 [\[ranges.syn\]](#)), `<span>` (22.7.2 [\[span.syn\]](#)), or `<utility>` (20.2.1 [\[utility.syn\]](#)) are included.

Edit 22.7.2 [\[span.syn\]](#) as indicated:

```
namespace std {  
    // constants  
    inline constexpr size_t dynamic_extent = numeric_limits<size_t>::max();
```

```

// [views.span], class template span
template<class ElementType, size_t Extent = dynamic_extent>
    class span;

template<class ElementType, size_t Extent>
    inline constexpr bool ranges::enable_safe_range<span<ElementType, Extent>> =
        true;

// [span.objectrep], views of object representation
template<class ElementType, size_t Extent>
    span<const byte, Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType)
        * Extent>
        as_bytes(span<ElementType, Extent> s) noexcept;

template<class ElementType, size_t Extent>
    span<byte, Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) *
        Extent>
        as_writable_bytes(span<ElementType, Extent> s) noexcept;

- // [span.tuple], tuple interface
- template<class T> struct tuple_size;
- template<size_t I, class T> struct tuple_element;
- template<class ElementType, size_t Extent>
-     struct tuple_size<span<ElementType, Extent>>;
- template<class ElementType>
-     struct tuple_size<span<ElementType, dynamic_extent>>; // not defined
- template<size_t I, class ElementType, size_t Extent>
-     struct tuple_element<I, span<ElementType, Extent>>;
- template<size_t I, class ElementType, size_t Extent>
-     constexpr ElementType& get(span<ElementType, Extent>) noexcept;
}

```

Delete 22.7.3.9 [\[span.tuple\]](#):

§ 22.7.3.9 Tuple interface [\[span.tuple\]](#)

```

template<class ElementType, size_t Extent>
struct tuple_size<span<ElementType, Extent>>
: integral_constant<size_t, Extent> { };

template<size_t I, class ElementType, size_t Extent>
struct tuple_element<I, span<ElementType, Extent>> {
using type = ElementType;
};

1 Mandates: Extent != dynamic_extent && I < Extent is true.

template<size_t I, class ElementType, size_t Extent>
constexpr ElementType& get(span<ElementType, Extent> s) noexcept;

2 Mandates: Extent != dynamic_extent && I < Extent is true.

3 Returns: s[I].

```

§ 3 References

[LWG3212] Tim Song. `tuple_element_t<1, const span<int, 42>>` is `const int`.

<https://wg21.link/lwg3212>

[N4849] Richard Smith. 2020. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4849>